Laboratorio 1: Auditoría de Seguridad

Objetivo: Evaluar la capacidad del alumno para detectar vulnerabilidades críticas generadas por IA y proponer mitigaciones arquitectónicas (S2S y FinOps).

1. El Problema (Entregable al Alumno)

Un desarrollador Junior usó IA para crear un sistema de pago de suscripción. La IA generó el siguiente componente en React:

```
import React, { useState } from 'react';
import { supabase } from './supabaseClient';

export default function BotonPago({ usuarioId, monto }) {
    const [cargando, setCargando] = useState(false);

    const procesarPago = async () => {
        setCargando(true);
        // Llamada a la API de pagos (Stripe/Transbank)
        const respuesta = await fetch('https://api.pagos.com/v1/charge', {
            method: 'POST',
            headers: {
                'Authorization': 'Bearer sk_live_super_secreta_12345',
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ user: usuarioId, amount: monto })
        });

        if (respuesta.ok) {
            // Actualizar el estado del usuario a "Premium"
            await supabase.from('users').update({ is_premium: true }).eq('id',
usuarioId);
            alert("Pago exitoso!");
        }
        setCargando(false);
    };

    return <button onClick={procesarPago} disabled={cargando}>Pagar ${monto}
</button>;
}
```

Misión del Alumno:

1. Identifica al menos 3 vulnerabilidades críticas (seguridad o financieras) en este código.
2. Usando la metodología PCA, redacta el *Prompt* exacto que le darías a la IA para refactorizar este sistema y hacerlo seguro.

2. Pauta de Evaluación (Exclusiva Docente)

El docente debe evaluar la respuesta del alumno buscando los siguientes puntos clave.

A. Identificación de Vulnerabilidades (3 Puntos)

El alumno debe mencionar:

1. **Fuga de API Key (1 pt):** La llave secreta (`sk_live...`) está hardcodeada en el Frontend. Cualquiera puede robarla desde *DevTools*.
2. **Manipulación del Cliente (1 pt):** Un atacante puede llamar a Supabase manualmente (`update({ is_premium: true })`) sin pagar. No hay seguridad de servidor.
3. **Falta de Rate Limiting / FinOps (1 pt):** Un bot puede hacer clic en el botón 10,000 veces por segundo, saturando la API y generando costos masivos.

B. Solución Arquitectónica / Prompt PCA (3 Puntos)

El prompt debe contener la exigencia de una arquitectura **Server-to-Server (S2S)**.

- **P**or qué: Explicar a la IA que la clave no puede estar en el cliente y la validación de DB debe ser segura.
- **C**ómo: Exigir la creación de una **Edge Function** (Backend) que reciba la petición de pago, valide, use la llave desde el `.env`, y actualice la base de datos *solo* si el pago fue validado por el servidor.
- **A** qué: Crear un nuevo archivo `/functions/pagos.ts` y limpiar el componente de React para que solo haga un fetch simple al backend propio.

> **Nota de Aprobación:** Si el alumno propone solucionar esto "encriptando la llave en React" o "desactivando el botón temporalmente", **reprueba**. La única solución es sacar la lógica al Backend.